QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control **3**

PRELIMINARY ROUND (Test Strategy & Planning) **3**

1. Scope & Requirements Traceability 3
2. Risk Assessment & Mitigation Strategy 3
3. Test Environment & Execution Strategy 5
4. CI/CD Release Thresholds & Automation Gates 5
5. Test Case Specifications (Drafts) 6
6. Test Strategy & Plan Sign-Off 7

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	*UMHackathon 2026 -**Balancia***
Team Repo URL	*https://github.com/aungmyintunsoe/balancia*
Project Board Link	*https://trello.com/b/EBMhIztF/fivegals*
Live Deployment URL (optional for preliminary round)	*https://balancia-liard.vercel.app/*

Objective:

The primary objective is to ensure that Balancia accepts vague goals in natural language, parses the intent into structured information through GLM, transforms it into the allocated microtasks according to the employees' skills, availability, and aspirations, detects overwork risks, and provides ROI analytics and insights.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

This section aligns testing with specific user requirements to ensure the testing is tailored according to the users' needs such that each test avoids any failure or bug related to the particular requirement, thereby preventing unexpected features from creeping up.

1.1 In-Scope Core Features

- **Goal Decomposition:** Manager input vague goals in natural language and GLM parse the intent into structured goals.
- **Task Allocation:** GLM assigns tasks to employees based on their skills, availability and aspiration.
- **Workload Dashboard:** Tracking employees's progress, productivity score and tasks completed
- **Overwork Detection:** System flags employees exceeding healthy workload
- **ROI analytics & Insights:** Calculate monthly average productivity and hour trend, team capacity, task status distribution and individual performance breakdown.

1.2 Out-of-Scope

- **Employee Chat**: Real chat between employees can be too complicated for MVP but can be implemented in the future
- **Mobile App**: Web only for hackathon MVP
- **Dark Mode**: Future enhancement

© UMHackathon 2026

Email: umhackathon@um.edu.my

- **Intelligence Support** for Employee: Future enhancement
- **Multi-language Support**: Currently English only; Chinese for future enhancement
- **Third-Party Integration**(Slack, Team): Future enhancement

2. Risk Assessment & Mitigation Strategy

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
AI allocates the tasks that don't match to the vague goal.	4	4	16 (Critical)	Set up strict rules for the AI to follow so it only gives the tasks that are related to the project.	Give AI the tricky goal to see whether it makes a mistake or asks for clarification.
AI assigning a coding task to a designer by mistake.	3	5	15 (high)	Make the system double-check the employee's skills and information in the database before confirming and assigning the task.	Try to assign a task that no one has that specific skill for and see if the system catches the error or not.
AI is giving tasks, but those are in wrong orders and timelines.	4	4	16 (Critical)	Add a code check that prevents "infinite loops" in the project timeline.	Create a goal with massive tasks, which are above 20+, to see if the timeline stays in the right order.
Information of Employees and their Work Performances gets leaked.	1	5	5 (Low)	Use strong passwords and encryption to keep all employee data hidden and safe.	Try to log in as a regular user and access the Manager's "Private Analytics" page.

Risk Assessment Scoring Criteria:

Likelihood (1-5)		*Severity (1-5)*	
1	*Rare*	*1*	*Impact is Negligible*
2	*Unlikely*	*2*	*Impact is Minor*

© UMHackathon 2026

Email: umhackathon@um.edu.my

3	*Possible*	*3*	*Moderate Impact*
4	*Likely*	*4*	*Major Impact*
5	*Almost Certain*	*5*	*Critical Failure of the system*

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	***Risk Level***	***Recommended Action***
1 – 5	*Low*	*Monitor only. Acceptable risks.*
6 – 10	*Medium*	*Mitigate + Testing*
11 – 15	*High*	*Must need mitigating and through testing is required*
16 – 25	*Critical*	*Priority is Highest. Need extensive level of testing.*

3. Test Environment & Execution Strategy

Our testing will take place in two main areas: **Localhost** (during development) and **Vercel Staging** (for final checks). We use **Mock data**, which is fake information, for employees and tasks to ensure we don't leak any real information. The testing is governed by a **"Critical-First"** rule, meaning the AI's core logic must be 100% stable before we focus on the visual part.

- **Unit Test**
  - Scope: AI Task Parsing
  - Execution: Executed locally by asserting fake task data response.
  - Isolation: We use "Mocking" to simulate the AI. This means we don't call the actual Z AI API every time; we just check if our code handles the *expected* text results correctly.
  - Pass Condition: The system must correctly accept the correct text structure and limit input to 1200 characters to conserve tokens and only allow valid input with fake tasks being generated properly.
- **Integration Test**
  - *Scope 1: The Core Orchestration*
  - *Execution: An Admin submits a valid*
  - *Workflow: We need to verify that Next.js calls ilmu.ai API from z.ai, receives the data and successfully inserts into the Supabase tasks table.*
  - *Pass Condition: Database reflects the change, and the UI updates the ongoing goals and displays the tasks.*

  - *Scope 2: Row level Security and Authentication*
  - *Execution: Trying to query other peoples data when logged in as another to check if set RLS policies are working as intended.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

- *Workflow: Manual Api calls to query data from supabase.*
- *Pass Condition:It prevents any data from being received when trying to access data from unauthorized rows.*

- *Scope 3: Employee interactivity*
- *Execution: Employee can see tasks assigned to them, be able to update them as started or finished, or an sos alert to submit a blocker reason.*
- *Workflow: Employees should be able to perform these functions and it should update in both database and for the admin.*
- *Pass Condition:It prevents any data from being received when looking at the console and sends the user back to the authentication page.*

- **Test Environment (CI/CD Practice):**
  - *Local Testing: This testing is manual one on localhost.*
  - *Staging/CI: Github actions allow for writing tests to find build or lint errors with each push.*

- **Regression Testing & Pass/Fail Rules:**
  - *Authentication and Session Persistence*
  - *Pass/Fail Condition: Unauthenticated users attempting to access the dashboard of another userID, must be routed successfully to the auth page. In the same browser, exiting and entering our website must remain in the signed in state, and skip the hero/ landing page.*

  - *Graceful Error handling*
    Pass/Fail Condition: If any ai API timesout or any server errors, it should not break the app or show the user an raw next.js error screen and must give an alert.

  - ***Continuation Rule:** While doing testing in a gradual manner, the initial cases must need to meet the threshold to allow continue for further testing*

- **Test Data Strategy:**
  - *Manual: We created 15+ fake profiles, with fake skills, and many organizations to start being able to test functions.*

- **Passing Rate Threshold:**
  - *We expect all tests to have a pass rate above 90%, and all tests regarding critical functions should pass 100% of the time, considering the leniency of the tests.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

4. CI/CD Release Thresholds & Automation Gates

4.1 Integration Thresholds (Merging to Main)

The primary needs of this threshold to be crucial is because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.

Checks	***Requirements***	***Project***	***Pass/Failed***
Automatic Build	***Zero Next.js Build Error***	***0 Errors***	***Passed***
Unit Tests	***100% Passing Rate for JSON Parsing***	***100%***	***Passed***
Code Quality	***Zero Linting Error when using github actions automation/ ErrorLens***	***100%***	***Passed***
Test Coverage	***Minimum 90%***	***100%***	***Passed***

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.2 Deployment Thresholds (Pushing to Production)

Checks	***Requirements***	***Project***	***Pass/Failed***
Regression Test	***Minimum 90%***	***100%***	***Passed***
AI Output Pass Rate	***Minimum of 90% valid response from API***	***100%***	***Passed***
Critical Bugs	***Zero P0/01 Bugs/ RLS bypasses***	***0***	***Passed***
API Performance	***Response Time < 800ms***	***586ms***	***Passed***
AI Orchestration Latency	***Returns payload within the 120s timeout guard***	***~10 - 45s depending on the complexity***	***Passed***
Security	***API keys are not exposed***	***Clean***	***Passed***

© UMHackathon 2026

Email: umhackathon@um.edu.my

5. Test Case Specifications (Drafts)

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	***Happy Case (Entire Flow):*** *Vague Goals to Allocating Tasks*	*Verify that the manager can input vague goals, GLM processes them to allocate tasks, and the manager can review distributed micro-tasks and details.*	*1. Open the website and log in with Gmail and password as admin.* *2. Manager goes to the Goals Page* *3. The manager types "Build a Networking Topology for APU Library" into the text area.* *4. Manager clicks the "Orchestrate WorkFlow" button.* *5. Wait for GLM response (5-30 seconds)* *6. Verify task distributions: description, assigned employees, estimated hours, and status*	*When the manager inputs the vague goals, the system processes and loads for around 15s and successfully allocates tasks based on employees' skills, availability, and aspiration.*	*Successfully transform from a vague goal into a structured goal, distribute the goal into micro-tasks, and allocate these tasks to the right employees. The overall process took 23 seconds.* *Status: Passed*
TC-02	***Specific Case (Negative):*** *GLM API Integration*	*Verify the system properly handles GLM API failure (e,g. APi unreachable, or invalid Key) and cannot perform intelligent task*	*1. Open the website and log in with Gmail and password as admin.* *2. Manager goes to the*	*Task allocation will fail and a timeout will occur when the GLM endpoint is unreachable. No fake tasks are*	*When the GLM API is unreachable, task allocation fails with 0/0 tasks, and the system does not*

© UMHackathon 2026

Email: umhackathon@um.edu.my

		allocation without GLM.	*Goals Page* *3. Temporarily make GLM API not responding* *4. Manager enters the vague goal into the text box* *5. Manager clicks the "Orchestrate WorkFlow" button.* *6. Wait for GLM response* *7. API call times out* *8. Observe system response* *9. Verify no task allocation is generated* *10. Verify system does not retry automatically*	*generated. The system does not crash.*	*show a retry button or it doesn't try to call the API again.* *Status: Passed (System correctly fails when GLM unavailable - proves GLM centrality)*
TC-03	***NFR (Performance):*** *API Response Test*	*AI API is relatively responsive(< 800ms) when multiple concurrent users are using it.*	*1. Open google developer tools under the network section.* *2. Do the goal generation and track multiple response times to figure out the average.*	*Average response time < 800ms in addition 0% error rate.*	*Average Response time is received as 586ms. 0% Error Rate.* *Status: Passed*

© UMHackathon 2026

Email: umhackathon@um.edu.my

6. AI Output & Boundary Testing (Drafts)

6.1. **Prompt/Response Test Pairs**

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	Build a new landing page for a coffee shop	Valid JSON with goals and tasks assigned to each employee accordingly to their skills	Parsed correctly, and insertions and ui updates are all correct and the tasks correspond with the employee's skills	Pass
AI-02	Deploy application on AWS	Valid JSON with goals and tasks assigned to each employee accordingly to their skills	Parsed correctly, and insertions and ui updates are all correct and the tasks correspond with the employee's skills	Pass
AI-03	Write a marketing campaign.	Valid JSON with goals and tasks assigned to each employee accordingly to their skills	Parsed correctly, and insertions and ui updates are all correct and the tasks correspond with the employee's skills	Pass

6.2. Oversized/Larger Input Test

In this section, do define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

Fields	Details
Maximum Input Size	1200 characters
Input used while testing	1500 characters of vague tasks

© UMHackathon 2026

Email: umhackathon@um.edu.my

Expected Behavior	The app catches the length, and returns a soft redirect error instead of hitting the AI.
Actual Behavior	System redirect as intended.
Status	Pass

6.3. Adversarial/Edge Prompt Test

We tested with an intentionally malicious input such as “DROP TABLE tasks;”
The Next.js backend forces strict JSON parsing on the AI response, so even if the AI outputs a malicious string, the output is only matched with columns and passed directly to Supabase. No raw query execution exists in the code base. The System safely caught the bad input and returned the error gracefully.

6.4. Hallucinating Handling

We made sure to extract strictly valid JSON objects and accept nothing else. And Supabase enforces schema strictness so that if any hallucinated ID by AI passes through, the database will throw an error and prevent corrupted data from entering the tables.

© UMHackathon 2026

Email: umhackathon@um.edu.my